

UNIVERSIDADE DE COIMBRA

Faculdade de Ciências e Tecnologia da Universidade de Coimbra



Googol

Motor de pesquisa de páginas Web

Trabalho realizado no âmbito da edição de 2025/2026 da disciplina de Sistemas Distribuídos da responsabilidade do docente Rogério Luís de Carvalho Costa do Departamento de Engenharia Informática da Licenciatura em Engenharia Informática da Faculdade de Ciências e Tecnologias da Universidade de Coimbra.

José Amado, José Silva

Coimbra, 2025

Índice

1. Introdução

- 1.1. Visão Geral do Projeto
- 1.2. Informações dos Autores
- 1.3. Organização do Projeto

2. Arquitetura do Projeto

- 2.1. Visão Geral da Arquitetura
- 2.2. Cliente
- 2.3. Gateway
 - 2.3.1. Visão Geral do Gateway
 - 2.3.2. Relação Gateway-Cliente
 - 2.3.3. Relação Gateway-Barrels
 - 2.3.4. Relação Gateway-URLQueue
 - 2.3.5. Relação Gateway-Downloaders
 - 2.3.6. Tolerância a Falhas
- 2.4. Barrels
 - 2.4.1. Visão Geral dos Barrels
 - 2.4.2. Esquema de Base de Dados e Índice Invertido
 - 2.4.3. Sistemas de Ordenação e Filtragem
 - 2.4.4. Protocolo de Sincronização Distribuída
 - 2.4.5. Tolerância a Falhas
- 2.5. URL Queue
 - 2.5.1. Visão Geral da URL Queue
 - 2.5.2. Fornecimento da Lista de Barrels aos Downloaders
- 2.6. Downloaders
 - 2.6.1. Visão Geral dos Downloaders
 - 2.6.2. Reliable Multicast e Tolerância de Falhas

3. Componente da Replicação Do Índice

- 3.1. Visão Geral da Replicação Do Índice
- 3.2. Algoritmo de Reliable Multicast
- 3.3. Protocolo de Sincronização

4. Componente RMI

- 4.1. Visão Geral do RMI
- 4.2. Registo e Descoberta de Serviços
- 4.3. Implementação de Callbacks

5. Testes Realizados

- 5.1. Tabela de Testes
- 5.2. Resultados e Observações

6. Distribuição de Tarefas

1.Introdução

1.1. Visão Geral

Este projeto implementa um motor de pesquisa web distribuído, capaz de indexar páginas, realizar pesquisas eficientes e garantir tolerância a falhas. O sistema é composto por vários módulos principais: gateway, storage barrels e downloaders, que comunicam entre si usando RPC/RMI e replicação.

1.2. Informações dos Autores

Este projeto foi realizado pelos seguintes alunos:

- José Gaspar Amado – 2023212241 – uc2023212241@student.uc.pt
- José Capinha Silva – 2023215540 – uc2023215540@student.uc.pt

1.3. Organização do Projeto

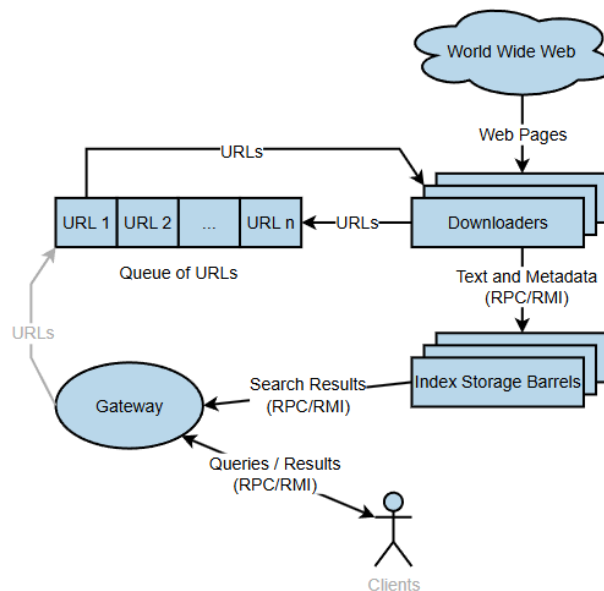
A organização das pastas do projeto é a seguinte:

- **/config:** Ficheiros de configuração para parametrização de componentes sem recompilação.
- **/data:** Dados persistentes (estatísticas e fila de urls) utilizados pelo sistema.
- **/docs:** Documentação técnica e relatório detalhado do projeto.
- **/misc:** Ficheiros auxiliares e documentação adicional.
- **/scripts:** Scripts relacionados às bases de dados utilizadas.
- **/src:** Código fonte.
- **/target:** Ficheiros compilados, binários e artefactos gerados.

2. Arquitetura Geral do Projeto

2.1. Visão Geral da Arquitetura

A arquitetura implementa o modelo distribuído especificado no enunciado, sendo composta por cinco componentes principais que comunicam via RMI: Cliente, Gateway, Storage Barrels, URL Queue e Downloaders. O sistema adota uma abordagem de microserviços onde cada componente tem responsabilidades bem definidas e comunica através de interfaces RMI.



2.2. Cliente

O cliente constitui a interface de utilizador do sistema, implementando uma aplicação de consola que permite aceder a todas as funcionalidades do motor de busca. Este componente estabelece comunicação exclusivamente com o Gateway através de RMI, suportando pesquisas com filtros avançados, indexação manual de URLs, consulta de backlinks e visualização de estatísticas do sistema em tempo real.

2.3. Gateway

2.3.1. Visão Geral do Gateway

O Gateway implementa um padrão de fachada que esconde a complexidade da arquitetura distribuída dos clientes, funcionando como ponto central de coordenação do sistema. Atua como intermediário entre clientes e outros componentes, agregando também estatísticas consolidadas de todas as operações do sistema.

2.3.2. Relação Gateway-Cliente

Os clientes estabelecem uma conexão RMI exclusivamente com o Gateway, nunca comunicando diretamente com outros componentes. Quando um cliente realiza uma pesquisa, o Gateway recebe a query e seleciona automaticamente um Barrel disponível através de round-robin.

2.3.3. Relação Gateway-Barrel

O Gateway mantém um registo dinâmico de todos os Barrels ativos. Após a sua inicialização, os Barrels registam-se automaticamente no Gateway, que aceita notificações de estado (ativo/inativo) e atualiza o registo interno correspondentemente. Este componente é também responsável pela sincronização dos Barrels.

2.3.4. Relação Gateway-URLQueue

O Gateway estabelece conexão com a URL Queue durante a inicialização. Quando um cliente submete um URL para indexação, o Gateway delega-o imediatamente para a URL Queue, marcando o URL como prioritário por ter sido submetido manualmente. O Gateway também notifica a URL Queue sobre mudanças de estado dos Barrels.

2.3.5. Relação Gateway-Downloaders

O Gateway não comunica diretamente com os Downloaders, mantendo uma separação clara de responsabilidades através da URL Queue como intermediário. Esta arquitetura permite que Downloaders sejam adicionados ou removidos dinamicamente sem afetar o Gateway além de eliminar a necessidade de criar um servidor RMI para os Downloaders.

2.3.6. Tolerância a Falhas

O Gateway implementa múltiplas camadas de tolerância a falhas, incluindo *retries* com *backoff* exponencial para comunicação com Barrels. Os Barrels não responsivos são automaticamente removidos e substituídos por outros disponíveis. Para a URL Queue, o Gateway continua operacional mesmo se a fila estiver indisponível, mantendo a funcionalidade de pesquisa.

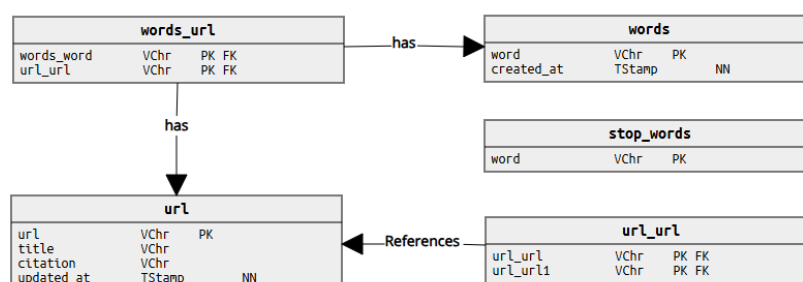
2.4. Barrels

2.4.1. Visão Geral dos Barrels

Os Index Storage Barrels constituem o núcleo de armazenamento do sistema, implementando réplicas idênticas que mantêm todo o índice invertido e metadados das páginas web. Cada Barrel é uma instância independente que executa numa porta específica e mantém uma base de dados PostgreSQL dedicada. A arquitetura dos Barrels segue o padrão de replicação ativa..

2.4.2. Esquema de Base de Dados e Índice Invertido

O sistema de cada Barrel utiliza PostgreSQL com um esquema otimizado para pesquisas eficientes. A tabela *words* armazena todas as palavras únicas encontradas durante indexação, enquanto *url* mantém metadados completos incluindo URL, título e citação extraída do conteúdo. O índice invertido é materializado através da tabela *words_url* que associa cada palavra aos URLs onde aparece. A tabela *url_url* implementa o grafo de ligações entre páginas. A tabela *stop_words* suporta o sistema de aprendizagem distribuída de palavras vazias, mantendo uma lista dinâmica de termos excluídos do índice.



2.4.3. Sistemas de Ordenação e Filtragem

Os resultados de pesquisa são automaticamente ordenados por relevância utilizando o número de backlinks como métrica principal. Os Barrels implementam um sistema de palavras vazias através de triggers PostgreSQL que aprendem automaticamente palavras muito comuns, e suportam pesquisas com múltiplos filtros como: domínio e alfabeto.

2.4.4. Protocolo de Sincronização Distribuída

Os Barrels participam num protocolo de sincronização coordenado pelo Gateway que garante consistência entre réplicas. O protocolo permite recuperação automática após falhas temporárias e garante que todas as réplicas convergem para um estado idêntico.

2.4.5. Tolerância a Falhas

Cada Barrel implementa mecanismos robustos de tolerância a falhas, incluindo retries automáticos. Em caso de falha durante operação, o Barrel tenta recuperação automática e notifica o Gateway sobre indisponibilidade temporária.

2.5. URL Queue

2.5.1. Visão Geral da URL Queue

A URL Queue constitui o componente central de coordenação para indexação distribuída, seguindo o padrão Producer-Consumer e funcionando como um buffer inteligente entre a recepção de URLs e o processamento pelos Downloaders. Implementa uma fila thread-safe que gere prioridades, evita duplicação, e distribui trabalho equitativamente entre Downloaders ativos.

2.5.2 Fornecimento da Lista de Barrels aos Downloaders

A URL Queue fornece aos Downloaders a lista atualizada de Barrels ativos, essencial para implementação do protocolo de *reliable multicast*. Esta lista é mantida atualizada através de notificações recebidas do Gateway sempre que ocorrem mudanças no estado dos Barrels.

2.6. Downloaders

2.6.1. Visão Geral dos Downloaders

Os Downloaders constituem o componente de *web crawling* do sistema, responsáveis por obter páginas web, extrair conteúdo e metadados, descobrir novos links, e distribuir dados processados para todos os Barrels através de multicast. Cada Downloader tem várias threads e executa *crawling* paralelo, maximizando o throughput de indexação. A arquitetura segue o padrão de *worker* distribuído, permitindo escalabilidade horizontal.

2.6.2. Reliable Multicast e Tolerância de Falhas

A distribuição de dados processados para os Barrels implementa um protocolo de reliable multicast construído sobre RMI. O Downloader distribui dados a todos os Barrels simultaneamente, garantindo que todas as réplicas recebem dados idênticos. O sistema requer confirmação de pelo menos 50% dos Barrels para considerar uma operação bem-sucedida.

3. Componente da Replicação Do Índice

3.1. Visão Geral da Replicação Do Índice

O sistema implementa replicação ativa do índice através de múltiplos Barrels que mantêm cópias idênticas de todos os dados garantindo alta disponibilidade e tolerância a falhas, permitindo que o sistema continue operacional mesmo com falhas simultâneas de múltiplos componentes. A replicação é gerida através de um protocolo de *reliable multicast* construído sobre RMI, onde os Downloaders distribuem dados para todas as réplicas simultaneamente e pela sincronização imposta pelo Gateway.

3.2. Algoritmo de Reliable Multicast

O algoritmo segue as seguintes etapas:

- **Obtenção da Lista de Destinos:** O Downloader obtém a lista atualizada de Barrels ativos através da URL Queue que retorna não apenas o próximo URL a processar mas também o mapa completo de Barrels disponíveis com suas referências RMI.
- **Multicast Paralelo:** Para cada página processada, o Downloader invoca simultaneamente todos os Barrels da lista. Cada invocação é feita de forma assíncrona, permitindo que falhas individuais não bloqueiem envio para outros Barrels.
- **Espera pelas Confirmações:** O sistema aguarda confirmações de todos os Barrels com timeout configurável. Barrels que não respondem dentro do timeout são considerados indisponíveis para esta operação.
- **Verificação:** O protocolo implementa uma política onde a operação é considerada válida se pelo menos 50% dos Barrels confirmarem recepção dos dados de modo a que dados nunca se percam mesmo com falhas simultâneas de múltiplos Barrels.
- **Tratamento de Falhas:** Barrels que falham são automaticamente removidos da lista local do Downloader e periodicamente testados para detectar a recuperação automática.

3.3. Protocolo de Sincronização

O sistema de sincronização segue as seguintes etapas:

- **Deteção de Inconsistências:** Cada Barrel calcula periodicamente um hash MD5 do seu índice e reporta ao Gateway. Quando hashes diferem entre Barrels, indica potencial inconsistência que requer sincronização.
- **Identificação de Dados em Falta:** O Gateway solicita a cada Barrel uma lista de URLs indexados e compara para identificar quais dados estão presentes em algumas réplicas mas ausentes noutras.
- **Transferência Seletiva:** Para URLs identificados como em falta, o Gateway coordena a transferência seletiva direta entre Barrels, permitindo sincronização eficiente sem duplicação desnecessária.

- **Validação:** Após a transferência, todos os Barrels recalculam hashes para confirmar que a sincronização foi bem-sucedida e que todas as réplicas convergem para estado idêntico.

4. Componente RMI

4.1. Visão Geral do RMI

O sistema utiliza três interfaces RMI principais que definem toda a comunicação distribuída: Gateway, Barrel e Url Queue interfaces.

- **GatewayInterface:** Define métodos para clientes e callbacks para componentes internos. Esta interface implementa o padrão fachada, expondo funcionalidade simplificada para clientes enquanto gere complexidade interna.
- **BarrelInterface:** Especifica operações de armazenamento. Todos os métodos são idempotentes, permitindo mais tentativas em caso de falhas.
- **UrlQueueInterface:** Define gestão da fila de URLs e fornecimento de listas de Barrels para coordenação do reliable multicast.

4.2. Registo e Descoberta de Serviços

O sistema utiliza RMI Registry para descoberta automática de serviços. O Gateway e URL Queue registam-se em portas específicas definidas no ficheiro *.properties* durante a inicialização. Os Barrels implementam auto-registo através de callback ao Gateway após a inicialização bem-sucedida. Esta abordagem permite descoberta dinâmica onde novos Barrels podem ser adicionados sem necessidade de reconfiguração de outros componentes.

4.3. Implementação de Callbacks

O sistema utiliza extensivamente callbacks RMI para comunicação assíncrona e notificação de eventos. Os callbacks implementados incluem:

- **reportBarrelStatus()**: Permite aos Barrels notificarem autonomamente o Gateway sobre mudanças de estado. O Gateway observa o estado dos Barrels através de notificações *event-driven*.
- **reportQueueStatus()**: Permite à URL Queue notificar autonomamente o Gateway após a sua inicialização e antes do seu termino.
- **reportIndexStats()**: Utilizado pelos Barrels para reportar estatísticas de índice atualizadas sempre que novos dados são adicionados pelos Downloaders.
- **reportSearchStats()**: Cada pesquisa processada por um Barrel gera um relatório com métricas de performance, permitindo ao Gateway calcular tempos médios de resposta discriminados por Barrel.

4.4. Solução de Failover

O failover é implementado através de múltiplas camadas. No nível cliente-Gateway, clientes implementam *retry* automático com tentativa de reestabelecer conexão em caso de *RemoteException*. No nível Gateway-Barrel, o Gateway mantém lista de Barrels ativos e automaticamente redireciona pesquisas para Barrels funcionais quando outros falham. Para operações críticas como reliable multicast, são consideradas bem-sucedidas se maioria das réplicas confirma receção.

5. Testes Realizados

5.1. Tabela de Testes

Teste	Descrição	Procedimento	Resultado Esperado	Resultado
1	Indexação manual e pesquisa básica	Iniciar sistema, indexar https://www.uc.pt , pesquisar 'universidade'.	URL indexado aparece nos resultados.	PASS
2	Failover automático de Barrel	Iniciar 2 Barrels + Gateway, terminar um, repetir pesquisa.	Gateway redireciona e pesquisa mantém-se funcional.	PASS
3	Stop words dinâmicas	Indexar mais de 100 páginas com 'the'.	Palavra 'the' removida automaticamente do índice.	PASS
4	Reliable multicast	Iniciar 2 Barrels, indexar urls.	Dados chegam a ambos.	PASS
5	Pesquisa com filtros avançados	Pesquisar 'tecnologia' sem e com filtro de domínio/idioma.	Filtros reduzem resultados mantendo relevância.	PASS
6	Sincronização automática entre Barrels	Indexar no Barrel A, iniciar Barrel B.	Barrel B sincroniza automaticamente via protocolo.	PASS
7	Recovery após falha de Gateway	Terminar Gateway durante pesquisa e reiniciar.	Cliente reconecta e operação repete com sucesso.	PASS
8	Paginação e ranking por relevância	Indexar 25 páginas, verificar paginação e ordem, verificar backlinks.	Resultados ordenados e paginação correta. Backlinks corretos	PASS
9	Crawling recursivo e descoberta de links	Indexar URL com vários links externos.	Novos links descobertos e processados automaticamente.	PASS
10	Load balancing e estatísticas	Executar 20 pesquisas com 2 Barrels ativos.	Distribuição equilibrada e estatísticas em tempo real.	PASS

5.2 Resultados e Observações

Após a realização dos testes verificou-se uma taxa de sucesso de 100% e registaram-se as relevantes observações:

- Sistema demonstra robustez completa conforme especificado no enunciado.
- Tolerância a falhas funciona transparentemente para utilizadores.
- Performance mantém-se consistente mesmo com falhas de componentes.
- Reliable multicast garante consistência entre todas as réplicas ativas.

6. Distribuição de tarefas

A distribuição de tarefas seguiu a opção sugerida no enunciado, onde cada elemento do grupo ficou responsável por componentes específicos do sistema distribuído. José Silva ficou responsável pelos Downloaders e pela componente multicast fiável dos Barrels, enquanto José Amado ficou responsável pelo Gateway e pela componente RMI dos Barrels.